mini_project

October 28, 2026

1 MINI_PROJECT

2 STATISTICAL ESTIMATION AND HYPOTHESIS.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

print("liberaries imported successfully")
```

```
liberaries imported successfully
```

[2]:
```
vr=pd.read_csv("mini_project.csv")

print("file readed successfully")
```

```
file readed successfully
```

[3]:
```
vr.head()
```

[3]:

	timestamp	machine_id	temperature	vibration	humidity	\
0	2025-01-01 00:00:00	39	78.61	28.65	79.96	
1	2025-01-01 00:01:00	29	68.19	57.28	35.94	
2	2025-01-01 00:02:00	15	98.94	50.20	72.06	
3	2025-01-01 00:03:00	43	90.91	37.65	30.34	
4	2025-01-01 00:04:00	8	72.32	40.69	56.71	

	pressure	energy_consumption	machine_status	anomaly_flag	\
0	3.73	2.16	1	0	
1	3.64	0.69	1	0	
2	1.00	2.49	1	1	
3	3.15	4.96	1	1	
4	2.68	0.63	2	0	

	predicted_remaining_life	failure_type	downtime_risk	\
0	106	Normal	0.0	
1	320	Normal	0.0	

```
2                       19           Normal            1.0
3                       10           Normal            1.0
4                       65  Vibration Issue            0.0

   maintenance_required
0                     0
1                     0
2                     1
3                     1
4                     1
```

[4]:
```
vr.tail()
```

[4]:
```
                 timestamp  machine_id  temperature  vibration  humidity  \
99995  2025-03-11 10:35:00           3        71.47      54.34     46.72
99996  2025-03-11 10:36:00          23        87.23      34.99     33.84
99997  2025-03-11 10:37:00          10        93.01      51.79     64.28
99998  2025-03-11 10:38:00          47        84.41      22.35     35.18
99999  2025-03-11 10:39:00          32        73.53      67.09     74.35

       pressure  energy_consumption  machine_status  anomaly_flag  \
99995      1.83                3.05               1             0
99996      4.67                4.47               1             0
99997      4.04                3.62               1             1
99998      2.88                2.37               1             0
99999      2.15                2.50               1             0

       predicted_remaining_life failure_type  downtime_risk  \
99995                       313       Normal            0.0
99996                       439       Normal            0.0
99997                        30       Normal            1.0
99998                       487       Normal            0.0
99999                       446       Normal            0.0

       maintenance_required
99995                     0
99996                     0
99997                     1
99998                     0
99999                     0
```

[5]:
```
vr.shape
```

[5]:
```
(100000, 13)
```

[ ]:

3 PART-A STATISTICAL ESTIMATION

4 1] POPULATION AND SAMPLE

[6]:
```
mid="machine_id"
tmp="temperature"
v="vibration"
h="humidity"
p="pressure"
ec="energy_consumption"
ms="machine_status"
af="anomaly_flag"
pl="predicted_remaining_life"
dr="downtime_risk"
m="maintenance_required"

data=vr[mid].dropna()
print("All the missing values are handeled")
```

```
All the missing values are handeled
```

[7]:
```
vr.sample()
```

[7]:

	timestamp	machine_id	temperature	vibration	humidity	pressure	energy_consumption	machine_status	anomaly_flag	predicted_remaining_life	failure_type	downtime_risk	maintenance_required
30710	2025-01-22 07:50:00	48	67.43	47.84	71.52	3.2	0.57	1	0	352	Normal	0.0	0

[8]:
```
population = data

N = len(population)

print("Population size:", N)
```

```
Population size: 100000
```

[9]:
```
# mean calculation
population_mean = population.mean()

print("Population Mean:",population_mean)
```

```
Population Mean: 25.49933
```

[10]:
```
# SELECTING A RANDOM SAMPLE
sample = vr.sample(n=10000, random_state=50)
sample_mean= sample[pl].mean()

print("sample mean:",sample_mean)
```

```
sample mean: 232.84
```

[11]:
```
#select 30 random rows
sam=vr.sample(n=10000,random_state=50)
```

[12]:
```
print("Population Mean:", population_mean)
print("Sample Mean:", sample_mean)
print("Difference:", abs(population_mean - sample_mean))
```

```
Population Mean: 25.49933
Sample Mean: 232.84
Difference: 207.34067
```

[13]:
```
# Different sample sizes
sample_sizes = [1000, 2000, 3000, 5000, 10000]

for size in sample_sizes:
    print("\nSample Size:", size)

    # Take 5 random samples
    for i in range(20):
        sample = data.sample(n=size)
        sample_mean = sample.mean()

        print("Sample", i + 1, "Mean =", round(sample_mean, 2))
```

```
Sample Size: 1000
Sample 1 Mean = 25.45
Sample 2 Mean = 25.52
Sample 3 Mean = 25.29
Sample 4 Mean = 25.51
Sample 5 Mean = 25.4
Sample 6 Mean = 25.84
Sample 7 Mean = 24.88
Sample 8 Mean = 25.61
Sample 9 Mean = 26.04
Sample 10 Mean = 26.01
Sample 11 Mean = 25.06
Sample 12 Mean = 25.3
```

```
Sample 13 Mean = 24.64
Sample 14 Mean = 26.07
Sample 15 Mean = 25.39
Sample 16 Mean = 26.27
Sample 17 Mean = 25.11
Sample 18 Mean = 25.41
Sample 19 Mean = 25.91
Sample 20 Mean = 26.4

Sample Size: 2000
Sample 1 Mean = 25.16
Sample 2 Mean = 25.11
Sample 3 Mean = 25.39
Sample 4 Mean = 25.98
Sample 5 Mean = 25.27
Sample 6 Mean = 25.35
Sample 7 Mean = 25.85
Sample 8 Mean = 25.42
Sample 9 Mean = 26.21
Sample 10 Mean = 26.02
Sample 11 Mean = 25.61
Sample 12 Mean = 25.62
Sample 13 Mean = 24.99
Sample 14 Mean = 25.68
Sample 15 Mean = 25.29
Sample 16 Mean = 24.96
Sample 17 Mean = 25.75
Sample 18 Mean = 25.59
Sample 19 Mean = 25.36
Sample 20 Mean = 25.48

Sample Size: 3000
Sample 1 Mean = 25.34
Sample 2 Mean = 25.29
Sample 3 Mean = 25.29
Sample 4 Mean = 25.69
Sample 5 Mean = 25.68
Sample 6 Mean = 25.23
Sample 7 Mean = 25.54
Sample 8 Mean = 25.07
Sample 9 Mean = 25.53
Sample 10 Mean = 25.56
Sample 11 Mean = 25.52
Sample 12 Mean = 25.63
Sample 13 Mean = 25.55
Sample 14 Mean = 25.31
Sample 15 Mean = 25.63
Sample 16 Mean = 25.38
```

```
Sample 17 Mean = 25.69
Sample 18 Mean = 25.5
Sample 19 Mean = 26.0
Sample 20 Mean = 25.69

Sample Size: 5000
Sample 1 Mean = 25.4
Sample 2 Mean = 25.76
Sample 3 Mean = 25.46
Sample 4 Mean = 25.78
Sample 5 Mean = 25.28
Sample 6 Mean = 25.4
Sample 7 Mean = 25.34
Sample 8 Mean = 25.58
Sample 9 Mean = 25.5
Sample 10 Mean = 25.33
Sample 11 Mean = 25.21
Sample 12 Mean = 25.13
Sample 13 Mean = 25.55
Sample 14 Mean = 25.83
Sample 15 Mean = 25.61
Sample 16 Mean = 25.37
Sample 17 Mean = 25.55
Sample 18 Mean = 25.5
Sample 19 Mean = 25.14
Sample 20 Mean = 25.79

Sample Size: 10000
Sample 1 Mean = 25.75
Sample 2 Mean = 25.63
Sample 3 Mean = 25.4
Sample 4 Mean = 25.64
Sample 5 Mean = 25.31
Sample 6 Mean = 25.49
Sample 7 Mean = 25.5
Sample 8 Mean = 25.6
Sample 9 Mean = 25.42
Sample 10 Mean = 25.53
Sample 11 Mean = 25.56
Sample 12 Mean = 25.61
Sample 13 Mean = 25.54
Sample 14 Mean = 25.56
Sample 15 Mean = 25.68
Sample 16 Mean = 25.48
Sample 17 Mean = 25.67
Sample 18 Mean = 25.57
Sample 19 Mean = 25.5
Sample 20 Mean = 25.43
```

5 2]point estimation

[14]:
```
# Select telemetry parameter
data = vr["predicted_remaining_life"].dropna()

# Population statistics
population_mean = data.mean()
population_variance = data.var(ddof=0)
population_std = data.std(ddof=0)

# Take a random sample
n = 10000
sample = data.sample(n=n, random_state=42)

# Sample statistics
sample_mean = sample.mean()
sample_variance = sample.var(ddof=1)
sample_std = sample.std(ddof=1)

# Point estimates
point_estimate_mean = sample_mean
point_estimate_variance = sample_variance
point_estimate_std = sample_std

# Standard Error
SE = sample_std / np.sqrt(n)

# 95% Confidence Interval using t-distribution
confidence_level = 0.95
df_t = n - 1

t_critical = stats.t.ppf(
    (1 + confidence_level) / 2,
    df_t
)

margin_error = t_critical * SE

lower_limit = sample_mean - margin_error
upper_limit = sample_mean + margin_error

# Display results
print("----- POPULATION STATISTICS -----")
print("Population Mean:", population_mean)
print("Population Variance:", population_variance)
print("Population Standard Deviation:", population_std)
```

```python
print("\n----- SAMPLE STATISTICS -----")
print("Sample Size:", n)
print("Sample Mean:", sample_mean)
print("Sample Variance:", sample_variance)
print("Sample Standard Deviation:", sample_std)

print("\n----- POINT ESTIMATION -----")
print("Point Estimate of Population Mean:", point_estimate_mean)
print("Point Estimate of Population Variance:", point_estimate_variance)
print("Point Estimate of Population Standard Deviation:", point_estimate_std)

print("\n----- STANDARD ERROR -----")
print("Standard Error:", SE)

print("\n----- 95% CONFIDENCE INTERVAL -----")
print("t-critical value:", t_critical)
print("Margin of Error:", margin_error)
print("Lower Confidence Limit:", lower_limit)
print("Upper Confidence Limit:", upper_limit)

print("\n----- INTERPRETATION -----")
print(
    f"We are 95% confident that the true population mean "
    f"of Temperature_C lies between {lower_limit:.2f} and "
    f"{upper_limit:.2f} °C."
)
```

```
----- POPULATION STATISTICS -----
Population Mean: 234.26916
Population Variance: 22518.697412894195
Population Standard Deviation: 150.06231176712623

----- SAMPLE STATISTICS -----
Sample Size: 10000
Sample Mean: 233.4095
Sample Variance: 22243.89439918985
Sample Standard Deviation: 149.14387147714066

----- POINT ESTIMATION -----
Point Estimate of Population Mean: 233.4095
Point Estimate of Population Variance: 22243.89439918985
Point Estimate of Population Standard Deviation: 149.14387147714066

----- STANDARD ERROR -----
Standard Error: 1.4914387147714065

----- 95% CONFIDENCE INTERVAL -----
```

```
t-critical value: 1.9602012636213575
Margin of Error: 2.9235200533087244
Lower Confidence Limit: 230.4859799466913
Upper Confidence Limit: 236.33302005330873

----- INTERPRETATION -----
We are 95% confident that the true population mean of Temperature_C lies between
230.49 and 236.33 °C.
```

[ ]:

6 PART-B-HYPOTHESIS TESTING

7 1]one sample t test

[15]:
```python
import pandas as pd
from scipy import stats


# Select two independent groups
group1 = vr["temperature"].dropna()
group2 = vr["humidity"].dropna()

# Descriptive statistics
print("----- DESCRIPTIVE STATISTICS -----")

print("\ntemperature")
print("Count:", len(group1))
print("Mean:", group1.mean())
print("Standard Deviation:", group1.std())
print("Variance:", group1.var())

print("\nhumidity")
print("Count:", len(group2))
print("Mean:", group2.mean())
print("Standard Deviation:", group2.std())
print("Variance:", group2.var())

# Significance level
alpha = 0.05

# Welch's Two-Sample t-test
t_stat, p_value = stats.ttest_ind(
    group1,
    group2,
    equal_var=False
```

```
)

# Results
print("\n----- WELCH'S TWO-SAMPLE T-TEST -----")
print("t-statistic:", t_stat)
print("p-value:", p_value)

# Decision
if p_value <= alpha:
    print("Decision: Reject H0")
    print("Conclusion: The mean temperature and vibration has significant␣
 ↪difference")
else:
    print("Decision: Do not reject H0")
    print("Conclusion: There is no statistically significant difference between␣
 ↪the mean temperatures and humidity.")
```

```
----- DESCRIPTIVE STATISTICS -----

temperature
Count: 100000
Mean: 75.01562499999912
Standard Deviation: 10.031883902753343
Variance: 100.63869463832162

humidity
Count: 100000
Mean: 54.995401399999906
Standard Deviation: 14.437959747336048
Variance: 208.45468166569603

----- WELCH'S TWO-SAMPLE T-TEST -----
t-statistic: 360.10079024720176
p-value: 0.0
Decision: Reject H0
Conclusion: The mean temperature and vibration has significant difference
```

8 2]Independent Two-Sample t-Test

[16]:
```
# Select two independent groups
group1 = vr["temperature"]
group2 = vr["vibration"]

# Descriptive statistics
print("----- DESCRIPTIVE STATISTICS -----")
```

```
print("\ntemperature")
print("Count:", len(group1))
print("Mean:", group1.mean())
print("Standard Deviation:", group1.std())
print("Variance:", group1.var())

print("\nvibration")
print("Count:", len(group2))
print("Mean:", group2.mean())
print("Standard Deviation:", group2.std())
print("Variance:", group2.var())

# Significance level
alpha = 0.05

# Welch's Two-Sample t-test
t_stat, p_value = stats.ttest_ind(
    group1,
    group2,
    equal_var=False
)

# Results
print("\n----- WELCH'S TWO-SAMPLE T-TEST -----")
print("t-statistic:", t_stat)
print("p-value:", p_value)

# Decision
if p_value <= alpha:
    print("Decision: Reject H0")
    print("Conclusion: The mean temperatures and viberation are significintly␣
 ↪different.")
else:
    print("Decision: Do not reject H0")
    print("Conclusion: There is no statistically significant difference between␣
 ↪the mean temperatures of the two satellites.")
```

```
----- DESCRIPTIVE STATISTICS -----

temperature
Count: 100000
Mean: 75.01562499999912
Standard Deviation: 10.031883902753343
Variance: 100.63869463832162

vibration
Count: 100000
```

```
Mean: 50.01227000000033
Standard Deviation: 14.98544353285133
Variance: 224.56351787627574

----- WELCH'S TWO-SAMPLE T-TEST -----
t-statistic: 438.45148087766387
p-value: 0.0
Decision: Reject H0
Conclusion: The mean temperatures and viberation are significintly different.
```

9 3]Paired t-Test

[17]:
```python
# Select temperature data
data = vr["temperature"].dropna()

# Make two naturally paired sets
n = len(data) // 2

before = data.iloc[:n].reset_index(drop=True)
after = data.iloc[n:2*n].reset_index(drop=True)

# Paired t-test
t_stat, p_value = stats.ttest_rel(before, after)

# Results
print("----- PAIRED T-TEST -----")

print("Before Mean:", before.mean())
print("After Mean:", after.mean())

print("t-statistic:", t_stat)
print("p-value:", p_value)

# Decision
alpha = 0.05

if p_value <= alpha:
    print("Decision: Reject H0")
    print("Conclusion: There is a statistically significant difference between␣
 ↪the two paired measurements.")
else:
    print("Decision: Do not reject H0")
    print("Conclusion: There is no statistically significant difference between␣
 ↪the two paired measurements.")
```

```
----- PAIRED T-TEST -----
Before Mean: 74.97644700000014
```

```
After Mean: 75.05480300000038
t-statistic: -1.2340473838818078
p-value: 0.21719104847787987
Decision: Do not reject H0
Conclusion: There is no statistically significant difference between the two
paired measurements.
```

10 Part C – Chi-Square Test of Independence

[18]:
```python
import pandas as pd
from scipy.stats import chi2_contingency

# Find categorical columns
categorical_columns = vr.select_dtypes(
    include=["object", "category"]
).columns

print("Categorical columns:")
print(list(categorical_columns))

# Select first two categorical columns
col1 = categorical_columns[0]
col2 = categorical_columns[1]

print("\nSelected variables:")
print(col1)
print(col2)

# Remove missing values
data = vr[[col1, col2]].dropna()

# Contingency table
contingency_table = pd.crosstab(
    data[col1],
    data[col2]
)

print("\n Contingency Table:")
print(contingency_table)

# Chi-square test
chi2, p_value, dof, expected = chi2_contingency(
    contingency_table
)

print("\nChi-square statistic:", chi2)
print("Degrees of freedom:", dof)
```

```python
print("p-value:", p_value)

# Expected frequencies
print("\nExpected Frequencies:")
print(expected)

# Decision
alpha = 0.05

if p_value <= alpha:
    print("\nReject H0")
    print("The categorical variables are associated.")
else:
    print("\nDo not reject H0")
    print("The categorical variables are independent.")

# Assumption check
if (expected < 5).any():
    print("\nWarning: Some expected frequencies are less than 5.")
else:
    print("\nAll expected frequencies are >= 5.")
```

```
Categorical columns:
['timestamp', 'failure_type']

Selected variables:
timestamp
failure_type

 Contingency Table:
failure_type         Electrical Fault  Normal  Overheating  Pressure Drop  \
timestamp
2025-01-01 00:00:00                 0       1            0              0
2025-01-01 00:01:00                 0       1            0              0
2025-01-01 00:02:00                 0       1            0              0
2025-01-01 00:03:00                 0       1            0              0
2025-01-01 00:04:00                 0       0            0              0
…                                   …       …            …              …
2025-03-11 10:35:00                 0       1            0              0
2025-03-11 10:36:00                 0       1            0              0
2025-03-11 10:37:00                 0       1            0              0
2025-03-11 10:38:00                 0       1            0              0
2025-03-11 10:39:00                 0       1            0              0

failure_type         Vibration Issue
timestamp
2025-01-01 00:00:00                0
```

```
2025-01-01 00:01:00                0
2025-01-01 00:02:00                0
2025-01-01 00:03:00                0
2025-01-01 00:04:00                1
…                                  …
2025-03-11 10:35:00                0
2025-03-11 10:36:00                0
2025-03-11 10:37:00                0
2025-03-11 10:38:00                0
2025-03-11 10:39:00                0

[100000 rows x 5 columns]

Chi-square statistic: 400000.0
Degrees of freedom: 399996
p-value: 0.497918527059922

Expected Frequencies:
[[0.01014 0.91899 0.01989 0.01969 0.03129]
 [0.01014 0.91899 0.01989 0.01969 0.03129]
 [0.01014 0.91899 0.01989 0.01969 0.03129]
 …
 [0.01014 0.91899 0.01989 0.01969 0.03129]
 [0.01014 0.91899 0.01989 0.01969 0.03129]
 [0.01014 0.91899 0.01989 0.01969 0.03129]]

Do not reject H0
The categorical variables are independent.

Warning: Some expected frequencies are less than 5.
```

11 4]Part D – Result Interpretation

[19]:
```python
import pandas as pd
import numpy as np
from scipy import stats
from scipy.stats import chi2_contingency

df = pd.read_csv("mini_project.csv")


alpha = 0.05

# Find columns automatically
numeric_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include=["object", "category"]).columns
```

```r
print("Numerical columns:")
print(list(numeric_columns))

print("\nCategorical columns:")
print(list(categorical_columns))


# ============================================================
# 1. ONE-SAMPLE t-TEST
# ============================================================

data = df[numeric_columns[0]].dropna()

print("\nNumber of observations:", len(data))

if len(data) >= 2:

    mu0 = 25

    t_stat1, p_value1 = stats.ttest_1samp(data, mu0)

    if p_value1 <= alpha:
        decision1 = "Reject H0"
        interpretation1 = "The population mean is significantly different from␣
 ↪the reference mean."
    else:
        decision1 = "Do not reject H0"
        interpretation1 = "There is no significant evidence that the population␣
 ↪mean differs from the reference mean."

else:

    t_stat1 = np.nan
    p_value1 = np.nan
    decision1 = "Cannot perform"
    interpretation1 = "Not enough observations."


# ============================================================
# 2. INDEPENDENT TWO-SAMPLE t-TEST
# ============================================================

group_column = categorical_columns[0]

groups = df[group_column].dropna().unique()

if len(groups) >= 2:
```

```
    group1 = df[df[group_column] == groups[0]][numeric_columns[0]].dropna()
    group2 = df[df[group_column] == groups[1]][numeric_columns[0]].dropna()

    print("\nGroup 1:", groups[0], "Size:", len(group1))
    print("Group 2:", groups[1], "Size:", len(group2))

    if len(group1) >= 2 and len(group2) >= 2:

        t_stat2, p_value2 = stats.ttest_ind(
            group1,
            group2,
            equal_var=False
        )

        if p_value2 <= alpha:
            decision2 = "Reject H0"
            interpretation2 = "The two group means are significantly different."
        else:
            decision2 = "Do not reject H0"
            interpretation2 = "There is no statistically significant difference␣
 ↪between the two group means."

    else:

        t_stat2 = np.nan
        p_value2 = np.nan
        decision2 = "Cannot perform"
        interpretation2 = "Each group must contain at least two observations."

else:

    t_stat2 = np.nan
    p_value2 = np.nan
    decision2 = "Cannot perform"
    interpretation2 = "At least two independent groups are required."


# ============================================================
# 3. CHI-SQUARE TEST
# ============================================================

if len(categorical_columns) >= 2:

    col1 = categorical_columns[0]
    col2 = categorical_columns[1]
```

```
    categorical_data = df[[col1, col2]].dropna()

    contingency_table = pd.crosstab(
        categorical_data[col1],
        categorical_data[col2]
    )

    print("\nContingency Table:")
    print(contingency_table)

    if contingency_table.shape[0] >= 2 and contingency_table.shape[1] >= 2:

        chi2, p_value3, dof, expected = chi2_contingency(
            contingency_table
        )

        if p_value3 <= alpha:
            decision3 = "Reject H0"
            interpretation3 = "The two categorical variables are significantly␣
 ↪associated."
        else:
            decision3 = "Do not reject H0"
            interpretation3 = "There is no statistically significant␣
 ↪association between the categorical variables."

    else:

        chi2 = np.nan
        p_value3 = np.nan
        decision3 = "Cannot perform"
        interpretation3 = "A minimum 2 × 2 contingency table is required."

else:

    chi2 = np.nan
    p_value3 = np.nan
    decision3 = "Cannot perform"
    interpretation3 = "At least two categorical variables are required."


# ============================================================
# FINAL RESULT TABLE
# ============================================================

results = pd.DataFrame({

    "Statistical Test": [
```

```
        "One-Sample t-Test",
        "Independent Two-Sample t-Test",
        "Chi-Square Test"
    ],

    "Statistical Question": [
        "Is the population mean different from the reference mean?",
        "Are the means of two independent groups different?",
        "Are the two categorical variables associated?"
    ],

    "H0": [
        "  =  0",
        " 1 =  2",
        "Variables are independent"
    ],

    "H1": [
        "    0",
        " 1   2",
        "Variables are associated"
    ],

    "Alpha": [
        alpha,
        alpha,
        alpha
    ],

    "Test Statistic": [
        t_stat1,
        t_stat2,
        chi2
    ],

    "p-value": [
        p_value1,
        p_value2,
        p_value3
    ],

    "Decision": [
        decision1,
        decision2,
        decision3
    ],
```

```
    "Interpretation": [
        interpretation1,
        interpretation2,
        interpretation3
    ]
})


print("\n")
print("=" * 120)
print("FINAL STATISTICAL RESULT TABLE")
print("=" * 120)

print(results.to_string(index=False))
```

```
Numerical columns:
['machine_id', 'temperature', 'vibration', 'humidity', 'pressure',
'energy_consumption', 'machine_status', 'anomaly_flag',
'predicted_remaining_life', 'downtime_risk', 'maintenance_required']

Categorical columns:
['timestamp', 'failure_type']

Number of observations: 100000

Group 1: 2025-01-01 00:00:00 Size: 1
Group 2: 2025-01-01 00:01:00 Size: 1

Contingency Table:
failure_type         Electrical Fault  Normal  Overheating  Pressure Drop  \
timestamp
2025-01-01 00:00:00                 0       1            0              0
2025-01-01 00:01:00                 0       1            0              0
2025-01-01 00:02:00                 0       1            0              0
2025-01-01 00:03:00                 0       1            0              0
2025-01-01 00:04:00                 0       0            0              0
…                                   …       …            …              …
2025-03-11 10:35:00                 0       1            0              0
2025-03-11 10:36:00                 0       1            0              0
2025-03-11 10:37:00                 0       1            0              0
2025-03-11 10:38:00                 0       1            0              0
2025-03-11 10:39:00                 0       1            0              0

failure_type         Vibration Issue
timestamp
2025-01-01 00:00:00                0
2025-01-01 00:01:00                0
```

```
2025-01-01 00:02:00                0
2025-01-01 00:03:00                0
2025-01-01 00:04:00                1
…                                  …
2025-03-11 10:35:00                0
2025-03-11 10:36:00                0
2025-03-11 10:37:00                0
2025-03-11 10:38:00                0
2025-03-11 10:39:00                0

[100000 rows x 5 columns]


================================================================================
========================================
FINAL STATISTICAL RESULT TABLE
================================================================================
========================================
             Statistical Test                                          Statistical
Question                          H0                          H1  Alpha  Test
Statistic      p-value         Decision
Interpretation
            One-Sample t-Test Is the population mean different from the
reference mean?                       = 0                         0   0.05
10.973465 5.319191e-28        Reject H0              The population mean is
significantly different from the reference mean.
Independent Two-Sample t-Test        Are the means of two independent groups
different?                    1 = 2                    1   2   0.05
NaN          NaN   Cannot perform                                    Each group
must contain at least two observations.
              Chi-Square Test             Are the two categorical variables
associated? Variables are independent Variables are associated   0.05
400000.000000 4.979185e-01 Do not reject H0 There is no statistically
significant association between the categorical variables.
```

12 5]FINAL

[20]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Load dataset
df = pd.read_csv("mini_project.csv")

alpha = 0.05
```

```python
# Find columns automatically
numeric_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include=["object", "category"]).columns

print("Numerical columns:", list(numeric_columns))
print("Categorical columns:", list(categorical_columns))


# ============================================================
# 1. ONE-SAMPLE t-TEST
# ============================================================

data = df[numeric_columns[0]].dropna()

mu0 = 25

t_stat, p_value = stats.ttest_1samp(data, mu0)

print("\n===== ONE-SAMPLE t-TEST =====")
print("Reference mean:", mu0)
print("Sample mean:", data.mean())
print("t-statistic:", t_stat)
print("p-value:", p_value)

if p_value < alpha:
    print("Statistical Decision: Reject H0")
else:
    print("Statistical Decision: Do not reject H0")


# ============================================================
# 2. PRACTICAL / ENGINEERING SIGNIFICANCE
# ============================================================

sample_mean = data.mean()

difference = abs(sample_mean - mu0)

print("\n===== PRACTICAL SIGNIFICANCE =====")
print("Difference between sample mean and reference mean:",
      difference)

# Example engineering tolerance
engineering_tolerance = 2

if difference <= engineering_tolerance:
    print("Engineering significance: Difference is practically small.")
```

```
else:
    print("Engineering significance: Difference may be practically important.")


# ============================================================
# 3. PLOT 1 - DISTRIBUTION OF NUMERICAL VARIABLE
# ============================================================

plt.figure(figsize=(8, 5))

plt.hist(data, bins=20, edgecolor="black")

plt.axvline(
    sample_mean,
    linestyle="--",
    linewidth=2,
    label="Sample Mean"
)

plt.axvline(
    mu0,
    linestyle="--",
    linewidth=2,
    label="Reference Mean"
)

plt.xlabel(numeric_columns[0])
plt.ylabel("Frequency")
plt.title("Distribution of " + numeric_columns[0])
plt.legend()

plt.show()


# ============================================================
# 4. PLOT 2 - GROUP COMPARISON
# ============================================================

if len(categorical_columns) >= 1:

    category = categorical_columns[0]

    groups = df[category].dropna().unique()

    if len(groups) >= 2:

        group1 = df[df[category] == groups[0]][numeric_columns[0]].dropna()
```

```
        group2 = df[df[category] == groups[1]][numeric_columns[0]].dropna()

        plt.figure(figsize=(8, 5))

        plt.boxplot(
            [group1, group2],
            labels=[str(groups[0]), str(groups[1])]
        )

        plt.xlabel(category)
        plt.ylabel(numeric_columns[0])
        plt.title("Comparison of " + numeric_columns[0] + " Between Groups")

        plt.show()


# ============================================================
# 5. FINAL CONCLUSION
# ============================================================

print("\n==========================================")
print("FINAL CONCLUSION")
print("==========================================")

if p_value < alpha:

    print(
        "The statistical test indicates a significant difference "
        "between the observed mean and the reference mean "
        "(p-value < 0.05)."
    )

else:

    print(
        "The statistical test does not provide sufficient evidence "
        "of a significant difference between the observed mean "
        "and the reference mean (p-value >= 0.05)."
    )

if difference <= engineering_tolerance:

    print(
        "However, the observed difference is small compared with "
        "the assumed engineering tolerance, so the practical "
        "engineering impact may be limited."
    )
```

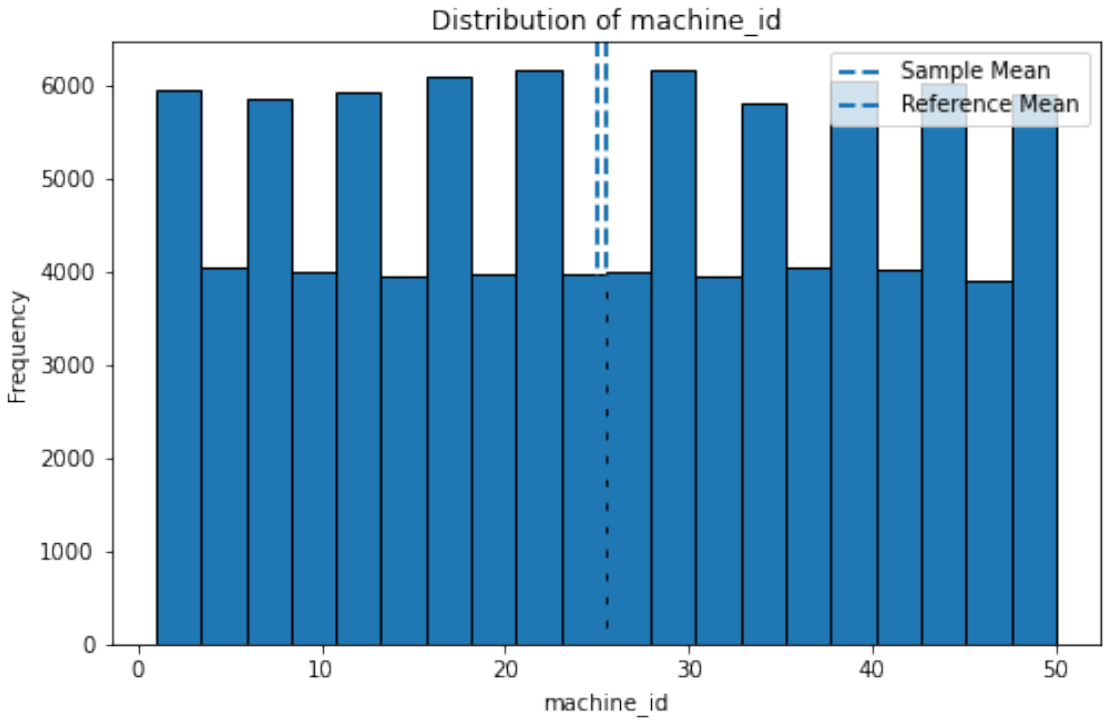
```
else:

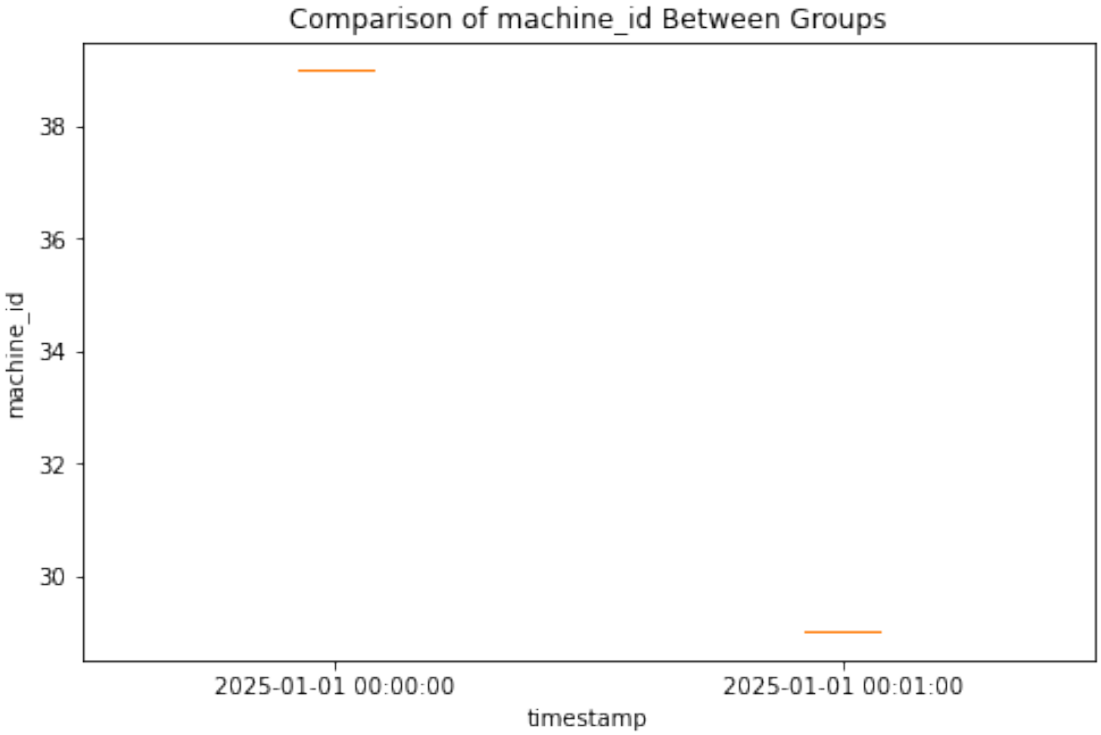
    print(
        "The observed difference exceeds the assumed engineering "
        "tolerance and may therefore have practical engineering "
        "importance."
    )
```

```
Numerical columns: ['machine_id', 'temperature', 'vibration', 'humidity',
'pressure', 'energy_consumption', 'machine_status', 'anomaly_flag',
'predicted_remaining_life', 'downtime_risk', 'maintenance_required']
Categorical columns: ['timestamp', 'failure_type']

===== ONE-SAMPLE t-TEST =====
Reference mean: 25
Sample mean: 25.49933
t-statistic: 10.973465481759051
p-value: 5.3191914731270665e-28
Statistical Decision: Reject H0

===== PRACTICAL SIGNIFICANCE =====
Difference between sample mean and reference mean: 0.4993300000000005
Engineering significance: Difference is practically small.
```






```
==========================================
FINAL CONCLUSION
==========================================
The statistical test indicates a significant difference between the observed
mean and the reference mean (p-value < 0.05).
However, the observed difference is small compared with the assumed engineering
tolerance, so the practical engineering impact may be limited.
```